

Practical – 1

```
# Import required libraries

import torch

import torch.nn as nn

import torch.optim as optim

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

# Step 1: Load dataset

df = pd.read_csv("student_study_data.csv")

print("Dataset Loaded Successfully!\n")

print(df.head())

# Step 2: Split input and output

X = df[['Hours_Studied', 'Attendance']]

y = df['Marks']

# Step 3: Split into train and test

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)

# Step 4: Scale input features

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

# Step 5: Convert to torch tensors

X_train = torch.tensor(X_train, dtype=torch.float32)

X_test = torch.tensor(X_test, dtype=torch.float32)

y_train = torch.tensor(y_train.values, dtype=torch.float32).view(-1, 1)

y_test = torch.tensor(y_test.values, dtype=torch.float32).view(-1, 1)

# Step 6: Define a simple neural network model

class StudentModel(nn.Module):

    def __init__(self):

        super(StudentModel, self).__init__()

        self.net = nn.Sequential(

            nn.Linear(2, 8), # Input layer (2 features)
```

```

        nn.ReLU(),
        nn.Linear(8, 4),
        nn.ReLU(),
        nn.Linear(4, 1)  # Output layer (Marks) )
    def forward(self, x):
        return self.net(x)
model = StudentModel()

# Step 7: Define loss function and optimizer
criterion = nn.MSELoss()      # Mean Squared Error loss
optimizer = optim.Adam(model.parameters(), lr=0.01)

# Step 8: Train the model
epochs = 300
for epoch in range(epochs):
    model.train()
    optimizer.zero_grad()
    outputs = model(X_train)
    loss = criterion(outputs, y_train)
    loss.backward()
    optimizer.step()
    if (epoch + 1) % 50 == 0:
        print(f'Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}')

# Step 9: Evaluate model on test data
model.eval()
with torch.no_grad():
    predictions = model(X_test)
    test_loss = criterion(predictions, y_test).item()
print("\nModel Evaluation Completed!")
print("Mean Squared Error (Test Loss):", round(test_loss, 4))

# Step 10: Show actual vs predicted marks
comparison = pd.DataFrame({
    "Actual Marks": y_test.flatten().numpy(),
    "Predicted Marks": predictions.flatten().numpy().round(2)
})

```

```
})  
print("\nPrediction Comparison:\n")  
print(comparison)
```

Output :

Dataset Loaded Successfully!

	Hours_Studied	Attendance	Marks
0	2	60	50
1	3	65	55
2	4	70	60
3	5	75	65
4	6	80	70

Epoch [50/300], Loss: 4958.9165
Epoch [100/300], Loss: 3308.8682
Epoch [150/300], Loss: 797.9849
Epoch [200/300], Loss: 42.7291
Epoch [250/300], Loss: 28.5550
Epoch [300/300], Loss: 20.0996

Model Evaluation Completed!

Mean Squared Error (Test Loss): 77.2457

Prediction Comparison:

	Actual Marks	Predicted Marks
0	60.0	51.270000
1	90.0	98.839996

Practical – 2

```
# a. Import the necessary packages

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Flatten

from tensorflow.keras.optimizers import SGD

from tensorflow.keras.datasets import mnist

import matplotlib.pyplot as plt

import numpy as np

# --- Configuration ---

NUM_CLASSES = 10

INPUT_SHAPE = (28, 28)

BATCH_SIZE = 128

EPOCHS = 10

LEARNING_RATE = 0.01

# b. Load the training and testing data (MNIST)

print("[INFO] Loading and preprocessing data...")

(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Preprocessing: Normalize the pixel values from 0-255 to 0-1

x_train = x_train.astype("float32") / 255.0

x_test = x_test.astype("float32") / 255.0

# The data is now (60000, 28, 28) and (10000, 28, 28).

# c. Define the network architecture using Keras (Sequential API)

# The Sequential model is a linear stack of layers

print("[INFO] Defining network architecture...")

model = Sequential([

    # The Flatten layer converts the 2D (28x28) image into a 1D vector (784)

    Flatten(input_shape=INPUT_SHAPE),

    # First Hidden Layer: 256 neurons with ReLU activation

    Dense(256, activation='relu'),

    # Second Hidden Layer: 128 neurons with ReLU activation

    Dense(128, activation='relu'),
```

```
# Output Layer: 10 neurons (one for each digit) with Softmax for probability distribution
Dense(NUM_CLASSES, activation='softmax'))

# Display the model structure
model.summary()

# d. Train the model using SGD

# 1. Compile the model

# The loss function is 'sparse_categorical_crossentropy' because the labels (y_train) are integers (0-9)
# The optimizer is Stochastic Gradient Descent (SGD)
print("[INFO] Compiling model with SGD optimizer...")
sgd_optimizer = SGD(learning_rate=LEARNING_RATE)
model.compile(
    optimizer=sgd_optimizer,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])

# 2. Fit the model to the training data
print("[INFO] Training network...")
H = model.fit(
    x_train, y_train,
    validation_data=(x_test, y_test),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=1 # Show progress bar)

# e. Evaluate the network
print("[INFO] Evaluating network...")
loss, accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test Loss: {loss:.4f}")
print(f"Test Accuracy: {accuracy*100:.2f}%")

# f. Plot the training loss and accuracy
print("[INFO] Plotting training loss and accuracy...")
plt.style.use("ggplot")
plt.figure(figsize=(12, 4))

# Plot Training Loss vs. Validation Loss
```

```
plt.subplot(1, 2, 1)
plt.plot(np.arange(0, EPOCHS), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, EPOCHS), H.history["val_loss"], label="val_loss")
plt.title("Training Loss and Validation Loss")
plt.xlabel("Epoch #")
plt.ylabel("Loss")
plt.legend()

# Plot Training Accuracy vs. Validation Accuracy
plt.subplot(1, 2, 2)
plt.plot(np.arange(0, EPOCHS), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, EPOCHS), H.history["val_accuracy"], label="val_acc")
plt.title("Training Accuracy and Validation Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Accuracy")
plt.legend()
plt.tight_layout()
plt.show()

# Example Prediction

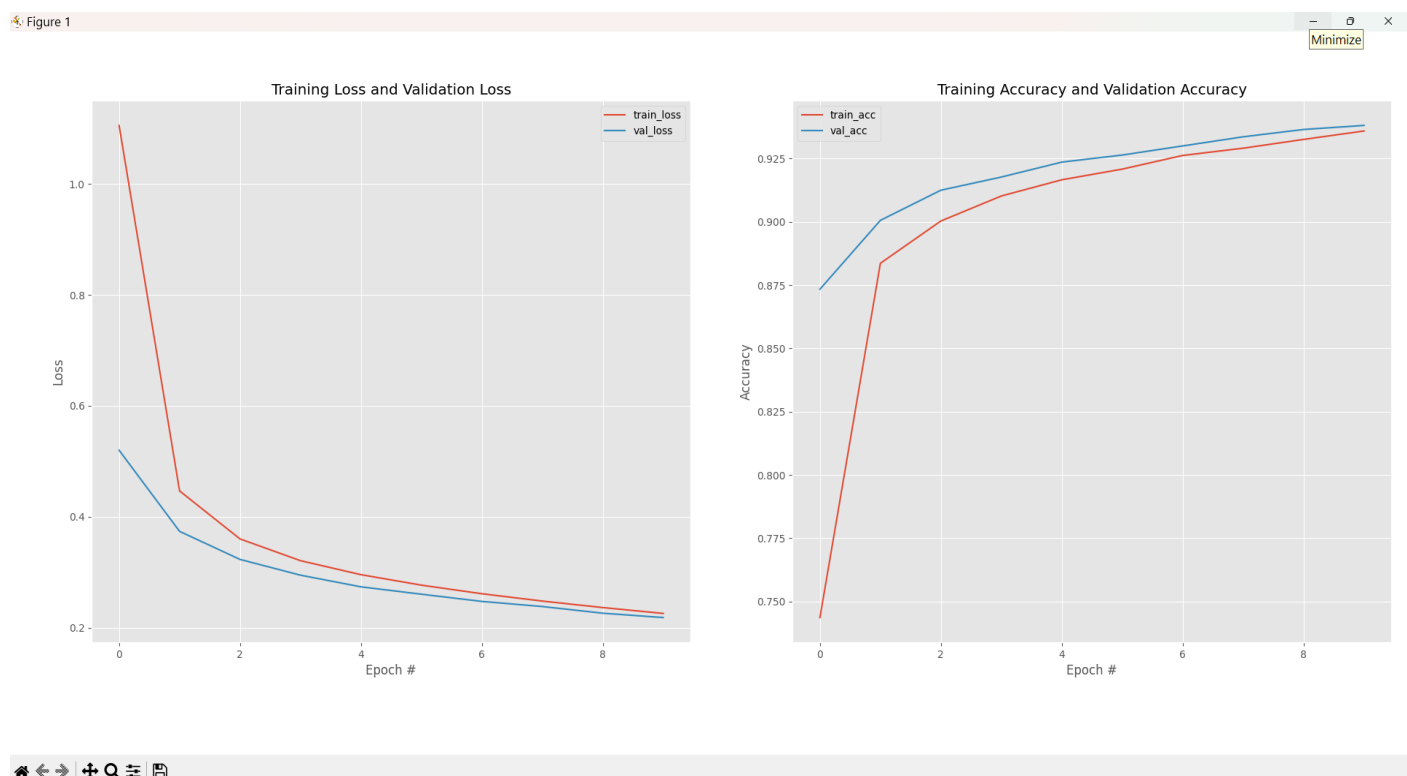
# Get a random image from the test set
random_index = np.random.randint(0, len(x_test))
sample_image = x_test[random_index]
true_label = y_test[random_index]

# Keras expects a batch of inputs, so we add a dimension
sample_image_batch = np.expand_dims(sample_image, axis=0)
prediction = model.predict(sample_image_batch)
predicted_label = np.argmax(prediction[0])
print(f"\nExample Prediction:")
print(f"True Label: {true_label}")
print(f"Predicted Label: {predicted_label}")
print(f"Confidence: {prediction[0][predicted_label]*100:.2f}%")
plt.imshow(sample_image, cmap='gray')
plt.title(f"True: {true_label}, Predicted: {predicted_label}")
```

plt.show()

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 256)	200,960
dense_1 (Dense)	(None, 128)	32,896
dense_2 (Dense)	(None, 10)	1,290

Total params: 235,146 (918.54 KB)
 Trainable params: 235,146 (918.54 KB)
 Non-trainable params: 0 (0.00 B)
 [INFO] Compiling model with SGD optimizer...
 [INFO] Training network...
 Epoch 1/10
 469/469 2s 4ms/step - accuracy: 0.7437 - loss: 1.1055 - val_accuracy: 0.8734 - val_loss: 0.5198
 Epoch 2/10
 469/469 2s 4ms/step - accuracy: 0.8836 - loss: 0.4465 - val_accuracy: 0.9006 - val_loss: 0.3736
 Epoch 3/10
 469/469 2s 4ms/step - accuracy: 0.9003 - loss: 0.3599 - val_accuracy: 0.9125 - val_loss: 0.3230
 Epoch 4/10
 469/469 2s 3ms/step - accuracy: 0.9102 - loss: 0.3207 - val_accuracy: 0.9177 - val_loss: 0.2946
 Epoch 5/10
 469/469 2s 5ms/step - accuracy: 0.9166 - loss: 0.2955 - val_accuracy: 0.9236 - val_loss: 0.2736
 Epoch 6/10
 469/469 2s 5ms/step - accuracy: 0.9208 - loss: 0.2766 - val_accuracy: 0.9264 - val_loss: 0.2603
 Epoch 7/10
 469/469 2s 4ms/step - accuracy: 0.9262 - loss: 0.2611 - val_accuracy: 0.9300 - val_loss: 0.2473
 Epoch 8/10
 469/469 2s 4ms/step - accuracy: 0.9291 - loss: 0.2479 - val_accuracy: 0.9336 - val_loss: 0.2381
 Epoch 9/10
 469/469 2s 4ms/step - accuracy: 0.9326 - loss: 0.2362 - val_accuracy: 0.9365 - val_loss: 0.2260
 Epoch 10/10
 469/469 2s 3ms/step - accuracy: 0.9359 - loss: 0.2256 - val_accuracy: 0.9381 - val_loss: 0.2182
 [INFO] Evaluating network...
 Test Loss: 0.2182
 Test Accuracy: 93.81%
 [INFO] Plotting training loss and accuracy...



Practical – 3

a. Loading and preprocessing the image data

```
import tensorflow as tf
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
# --- 1. CONFIGURATION ---
```

```
NUM_CLASSES = 10
```

```
EPOCHS = 10
```

```
BATCH_SIZE = 128
```

```
print("--- Starting CNN Image Classification ---")
```

a. Loading and preprocessing the image data

```
print("\n[Stage A] Loading and preprocessing image data...")
```

```
# Load MNIST data
```

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
```

```
# Preprocessing Step 1: Reshape data for CNN input
```

```
# Images must be 4D: (samples, height, width, channels). Grayscale = 1 channel.
```

```
x_train = x_train.reshape(x_train.shape[0], 28, 28, 1)
```

```
x_test = x_test.reshape(x_test.shape[0], 28, 28, 1)
```

```
input_shape = (28, 28, 1)
```

```
# Preprocessing Step 2: Normalize pixel values (0-255 -> 0.0-1.0)
```

```
x_train = x_train.astype("float32") / 255.0
```

```
x_test = x_test.astype("float32") / 255.0
```

```
print(f"Input shape: {input_shape}")
```

b. Defining the model's architecture

```
print("\n[Stage B] Defining the model's architecture (CNN)...")
```

```
model = Sequential([
```

```
    # 1. Convolutional Block 1
```

```
    Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=input_shape),
```

```
    MaxPooling2D(pool_size=(2, 2)),
```

```
    # 2. Convolutional Block 2
```



```

Conv2D(64, kernel_size=(3, 3), activation='relu'),
MaxPooling2D(pool_size=(2, 2)),
# 3. Classification Head
Flatten(), # Prepare output for Dense layers
Dense(128, activation='relu'),
Dense(NUM_CLASSES, activation='softmax') # Output layer
])
model.summary()
# c. Training the model
print("\n[Stage C] Compiling and training the model...")
# Compile the model using a popular optimizer and loss function
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy', # Used for integer labels (0-9)
    metrics=['accuracy']
)
# Fit the model to the training data
H = model.fit(
    x_train, y_train,
    validation_data=(x_test, y_test),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=1
)
print("Model training complete.")
# d. Estimating the model's performance
print("\n[Stage D] Estimating the model's performance...")
# Evaluate model on the test dataset
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f'Final Test Loss: {test_loss:.4f}')
print(f'Final Test Accuracy: {test_accuracy*100:.2f}%')
# Plotting the Training History

```

```
plt.style.use("ggplot")
plt.figure(figsize=(12, 5))

# Plot Loss
plt.subplot(1, 2, 1)
plt.plot(np.arange(0, EPOCHS), H.history["loss"], label="Train Loss")
plt.plot(np.arange(0, EPOCHS), H.history["val_loss"], label="Validation Loss")
plt.title("Training and Validation Loss")
plt.xlabel("Epoch #")
plt.ylabel("Loss")
plt.legend()

# Plot Accuracy
plt.subplot(1, 2, 2)
plt.plot(np.arange(0, EPOCHS), H.history["accuracy"], label="Train Accuracy")
plt.plot(np.arange(0, EPOCHS), H.history["val_accuracy"], label="Validation Accuracy")
plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Accuracy")
plt.legend()

plt.tight_layout()
plt.show()

# Example Prediction
random_index = np.random.randint(0, len(x_test))
sample_image = x_test[random_index]
true_label = y_test[random_index]

# Predict requires a batch, so we add a dimension (1, 28, 28, 1)
sample_image_batch = np.expand_dims(sample_image, axis=0)
prediction = model.predict(sample_image_batch, verbose=0)
predicted_label = np.argmax(prediction[0])

print(f"\nExample Prediction on Test Image:")
print(f"True Label: {true_label}")
print(f"Predicted Label: {predicted_label}")
```

Output :

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)

[Stage C] Compiling and training the model...

Epoch 1/10
469/469 10s 20ms/step - accuracy: 0.9398 - loss: 0.2065 - val_accuracy: 0.9830 - val_loss: 0.0560
Epoch 2/10
469/469 15s 33ms/step - accuracy: 0.9827 - loss: 0.0570 - val_accuracy: 0.9866 - val_loss: 0.0436
Epoch 3/10
469/469 15s 32ms/step - accuracy: 0.9872 - loss: 0.0406 - val_accuracy: 0.9879 - val_loss: 0.0375
Epoch 4/10
469/469 15s 32ms/step - accuracy: 0.9904 - loss: 0.0308 - val_accuracy: 0.9887 - val_loss: 0.0364
Epoch 5/10
469/469 15s 32ms/step - accuracy: 0.9925 - loss: 0.0246 - val_accuracy: 0.9896 - val_loss: 0.0317
Epoch 6/10
469/469 15s 33ms/step - accuracy: 0.9939 - loss: 0.0190 - val_accuracy: 0.9882 - val_loss: 0.0368

Epoch 7/10
469/469 15s 32ms/step - accuracy: 0.9953 - loss: 0.0159 - val_accuracy: 0.9900 - val_loss: 0.0336
Epoch 8/10
Epoch 8/10
469/469 15s 32ms/step - accuracy: 0.9964 - loss: 0.0119 - val_accuracy: 0.9911 - val_loss: 0.0300
469/469 15s 32ms/step - accuracy: 0.9964 - loss: 0.0119 - val_accuracy: 0.9911 - val_loss: 0.0300
Epoch 9/10
469/469 12s 25ms/step - accuracy: 0.9965 - loss: 0.0102 - val_accuracy: 0.9913 - val_loss: 0.0311
Epoch 10/10
469/469 8s 18ms/step - accuracy: 0.9968 - loss: 0.0095 - val_accuracy: 0.9928 - val_loss: 0.0309
Model training complete.

[Stage D] Estimating the model's performance...

Final Test Loss: 0.0309
Final Test Accuracy: 99.28%

Example Prediction on Test Image:
True Label: 4
Predicted Label: 4



Practical – 4

```
# a. Import required libraries

import tensorflow as tf

from tensorflow.keras.models import Model

from tensorflow.keras.layers import Input, Dense

from tensorflow.keras.optimizers import Adam

from tensorflow.keras.losses import MeanSquaredError

import numpy as np

import matplotlib.pyplot as plt

# Define constants

ANOMALY_DIGIT = 9 # Digit 9 will be treated as the anomaly

IMAGE_SIZE = 28 * 28

LATENT_DIM = 32 # The size of the compressed bottleneck layer

EPOCHS = 20

BATCH_SIZE = 128

print("--- Starting Autoencoder Anomaly Detection ---")

# b. Upload/access the dataset

print("\n[Stage A] Accessing and preprocessing dataset...")

# Load MNIST data

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalize and Flatten the image data (28x28 -> 784 features)

x_train = x_train.astype('float32') / 255.0

x_test = x_test.astype('float32') / 255.0

x_train = x_train.reshape((len(x_train), IMAGE_SIZE))

x_test = x_test.reshape((len(x_test), IMAGE_SIZE))

# Prepare 'Normal' data (digits 0-8) and 'Anomaly' data (digit 9)

x_train_normal = x_train[y_train != ANOMALY_DIGIT]

x_test_normal = x_test[y_test != ANOMALY_DIGIT]

x_test_anomaly = x_test[y_test == ANOMALY_DIGIT]

# Create a balanced test subset for final evaluation and visualization

x_test_subset = np.concatenate([x_test_normal[:100], x_test_anomaly[:100]])
```

```

y_test_subset = np.concatenate([y_test[y_test != ANOMALY_DIGIT][:100], y_test[y_test ==
ANOMALY_DIGIT][:100]])

# c. The encoder converts it into a latent representation
print("\n[Stage B] Defining Encoder Network...")
input_layer = Input(shape=(IMAGE_SIZE,))
# Encoder Architecture (compression)
encoder = Dense(128, activation='relu')(input_layer)
encoder = Dense(64, activation='relu')(encoder)
# Latent representation (bottleneck)
latent_view = Dense(LATENT_DIM, activation='relu', name='latent_space')(encoder)
# d. Decoder networks convert it back to the original input
print("[Stage C] Defining Decoder Network...")
# Decoder Architecture (reconstruction)
decoder = Dense(64, activation='relu')(latent_view)
decoder = Dense(128, activation='relu')(decoder)
# Output layer (must reconstruct the original 784 pixels, using sigmoid for 0-1 range)
output_layer = Dense(IMAGE_SIZE, activation='sigmoid')(decoder)
# Combine Encoder and Decoder into the Autoencoder Model
autoencoder = Model(inputs=input_layer, outputs=output_layer, name='Autoencoder')
autoencoder.summary()
# e. Compile the models with Optimizer, Loss, and Evaluation Metrics
print("\n[Stage D] Compiling and training the Autoencoder...")
# Compile the model
autoencoder.compile(
    optimizer=Adam(learning_rate=0.001),
    loss=MeanSquaredError() # Standard loss for reconstruction)
# Train ONLY on 'normal' data (digits 0-8)
history = autoencoder.fit(
    x_train_normal, x_train_normal, # Input == Target (unsupervised)
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    shuffle=True,

```

```

validation_data=(x_test_normal, x_test_normal),
verbose=1)

print("\n--- Anomaly Detection Evaluation ---")

# --- Performance Estimation: Calculating Reconstruction Error ---

# 1. Predict reconstruction for the test subset
predictions = autoencoder.predict(x_test_subset, verbose=0)

# 2. Calculate the Mean Squared Error (Reconstruction Error) for each sample
mse = np.mean(np.power(x_test_subset - predictions, 2), axis=1)

# 3. Find a threshold: Use the 95th percentile of the MSE on the normal test set.
mse_normal_test = np.mean(np.power(x_test_normal - autoencoder.predict(x_test_normal, verbose=0), 2),
axis=1)

threshold = np.percentile(mse_normal_test, 95)

# 4. Classify anomalies
is_anomaly = mse > threshold

# 5. Display detection stats
print(f'Reconstruction Error Threshold (95th percentile of normal data): {threshold:.5f}')
print(f'Total test samples evaluated: {len(x_test_subset)}')
print(f'True Anomaly (Digit {ANOMALY_DIGIT}) count: {np.sum(y_test_subset ==
ANOMALY_DIGIT)}')
print(f'Detected Anomaly count (MSE > Threshold): {np.sum(is_anomaly)}')

# 6. Visualize results (with safe indexing)
# Get indices for correctly detected anomalies (True Anomaly & Error > Threshold)
anomaly_indices = np.where(np.logical_and(is_anomaly, y_test_subset == ANOMALY_DIGIT))[0]
# Get indices for correctly classified normal samples (True Normal & Error < Threshold)
normal_indices = np.where(np.logical_and(~is_anomaly, y_test_subset != ANOMALY_DIGIT))[0]
# Determine the number of plots possible (max 5)
num_plots = min(5, len(anomaly_indices), len(normal_indices))
if num_plots > 0:
    fig, axes = plt.subplots(2, num_plots, figsize=(3 * num_plots, 6))
    # Check if axes is a 1D array (only possible if num_plots=1, but safe indexing handles this)
    is_single_plot = num_plots == 1
    for i in range(num_plots):
        idx_anomaly = anomaly_indices[i]

```

```

idx_normal = normal_indices[i]

# Determine correct axis handles
ax0 = axes[0] if is_single_plot else axes[0, i]
ax1 = axes[1] if is_single_plot else axes[1, i]

# Plot Anomaly (Digit 9)
ax0.imshow(x_test_subset[idx_anomaly].reshape(28, 28), cmap='gray')
ax0.set_title(f'Anomaly (9)\nError: {mse[idx_anomaly]:.4f}')
ax0.axis('off')

# Plot Normal (Digit 0-8)
ax1.imshow(x_test_subset[idx_normal].reshape(28, 28), cmap='gray')
ax1.set_title(f'Normal\nError: {mse[idx_normal]:.4f}')
ax1.axis('off')

plt.suptitle(f'Anomaly Detection using Autoencoder (Trained on 0-8, Anomaly is 9)\nThreshold:
{threshold:.5f}')

plt.tight_layout()

plt.show()

else:

    print("\nNot enough correctly classified anomaly/normal samples to generate plot.")

```

Output :

Model: "Autoencoder"		
Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
latent_space (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 64)	2,112
dense_3 (Dense)	(None, 128)	8,320
dense_4 (Dense)	(None, 784)	101,136

Total params: 222,384 (868.69 KB)
 Trainable params: 222,384 (868.69 KB)
 Non-trainable params: 0 (0.00 B)

[Stage D] Compiling and training the Autoencoder...

Epoch 1/20
 423/423 5s 6ms/step - loss: 0.0506 - val_loss: 0.0281

Epoch 2/20
 423/423 3s 6ms/step - loss: 0.0241 - val_loss: 0.0210

Epoch 3/20
 423/423 3s 7ms/step - loss: 0.0198 - val_loss: 0.0179

Epoch 4/20
 423/423 3s 8ms/step - loss: 0.0171 - val_loss: 0.0158

Epoch 5/20
 423/423 7s 11ms/step - loss: 0.0153 - val_loss: 0.0144

Epoch 6/20
 423/423 5s 11ms/step - loss: 0.0141 - val_loss: 0.0133

```

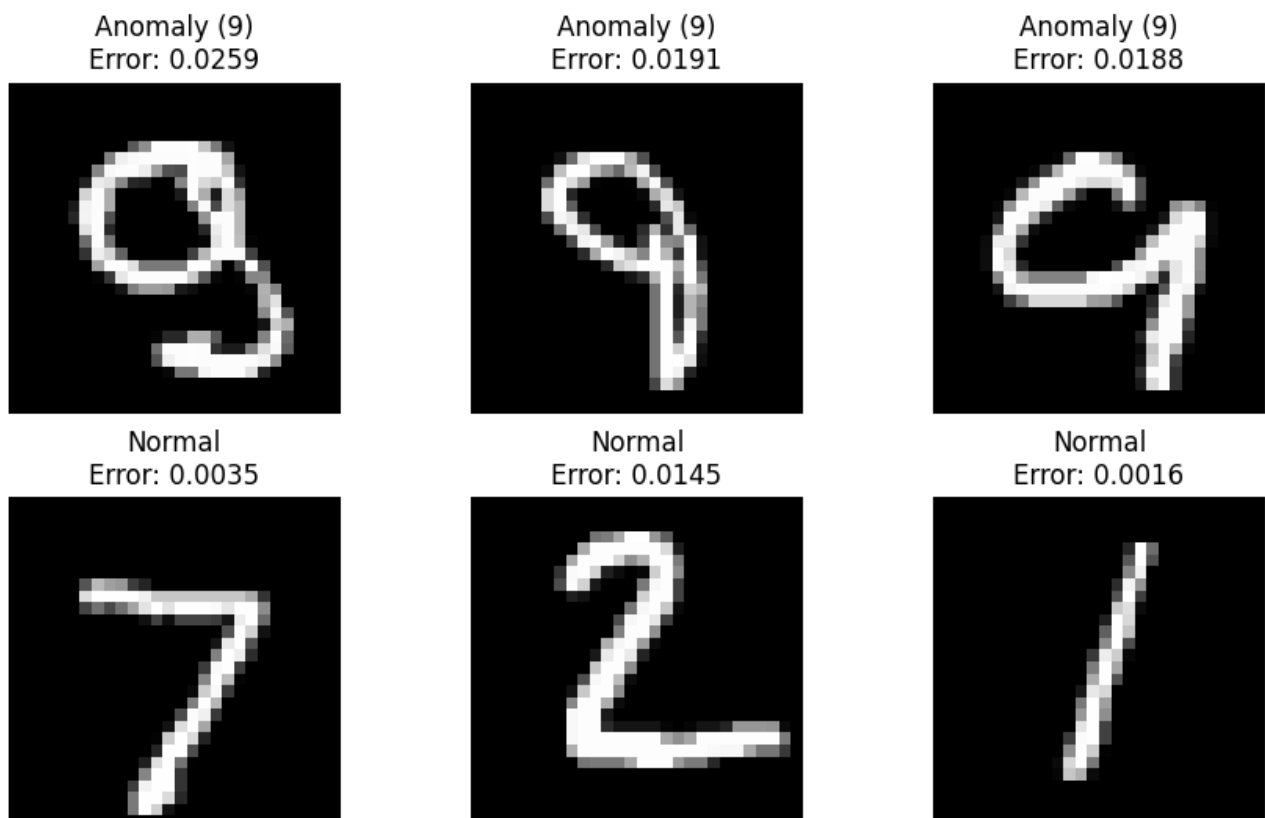
Epoch 7/20
423/423 ██████████ 5s 11ms/step - loss: 0.0133 - val_loss: 0.0126
Epoch 8/20
423/423 ██████████ 3s 8ms/step - loss: 0.0125 - val_loss: 0.0119
Epoch 9/20
423/423 ██████████ 5s 7ms/step - loss: 0.0119 - val_loss: 0.0114
Epoch 10/20
423/423 ██████████ 2s 5ms/step - loss: 0.0114 - val_loss: 0.0110
Epoch 11/20
423/423 ██████████ 3s 6ms/step - loss: 0.0110 - val_loss: 0.0106
Epoch 12/20
423/423 ██████████ 3s 6ms/step - loss: 0.0106 - val_loss: 0.0106
Epoch 13/20
423/423 ██████████ 2s 5ms/step - loss: 0.0102 - val_loss: 0.0099
Epoch 14/20
423/423 ██████████ 3s 7ms/step - loss: 0.0099 - val_loss: 0.0097
Epoch 15/20
423/423 ██████████ 5s 11ms/step - loss: 0.0096 - val_loss: 0.0095
Epoch 16/20
423/423 ██████████ 5s 11ms/step - loss: 0.0094 - val_loss: 0.0092
Epoch 17/20
423/423 ██████████ 5s 11ms/step - loss: 0.0092 - val_loss: 0.0091
Epoch 18/20
423/423 ██████████ 3s 6ms/step - loss: 0.0091 - val_loss: 0.0090
Epoch 19/20
423/423 ██████████ 3s 6ms/step - loss: 0.0089 - val_loss: 0.0088
Epoch 20/20
423/423 ██████████ 3s 7ms/step - loss: 0.0087 - val_loss: 0.0085

--- Anomaly Detection Evaluation ---
Reconstruction Error Threshold (95th percentile of normal data): 0.01803
Total test samples evaluated: 200
True Anomaly (Digit 9) count: 100
Detected Anomaly count (MSE > Threshold): 10

```

Figure 1

Anomaly Detection using Autoencoder (Trained on 0-8, Anomaly is 9)
Threshold: 0.01803



Practical – 5

```
# a. Data preparation

import numpy as np

import tensorflow as tf

from tensorflow.keras.models import Model

from tensorflow.keras.layers import Input, Dense, Embedding, Lambda, Reshape

from tensorflow.keras.optimizers import Adam

from tensorflow.keras.utils import to_categorical

from tensorflow.keras import backend as K

# --- Configuration ---

CONTEXT_WINDOW_SIZE = 2 # Two words before and two words after

EMBEDDING_DIM = 50      # Dimensionality of the word vectors

HIDDEN_DIM = 128

BATCH_SIZE = 32         # <-- FIX: Added missing BATCH_SIZE definition

EPOCHS = 100

LEARNING_RATE = 0.001

def create_lookup_tables(text_data):

    """Tokenizes text and creates word-to-index and index-to-word mappings."""

    # Simple tokenization: lowercasing and splitting by space

    tokens = [word.lower() for sentence in text_data for word in sentence.split()]

    # Vocabulary creation

    vocab = sorted(list(set(tokens)))

    word_to_idx = {word: i for i, word in enumerate(vocab)}

    idx_to_word = {i: word for i, word in enumerate(vocab)}

    # Vocabulary size

    vocab_size = len(vocab)

    return tokens, vocab_size, word_to_idx, idx_to_word

# Sample Corpus

text_data = [

    "The quick brown fox jumps over the lazy dog",

    "Word embeddings are useful for natural language processing",

    "CBOW predicts the word from context"]
```

```

print("--- Starting CBOW Implementation ---")
print("\n[Stage A] Data Preparation...")
tokens, VOCAB_SIZE, word_to_idx, idx_to_word = create_lookup_tables(text_data)
print(f'Vocabulary Size: {VOCAB_SIZE}')
print(f'Total Tokens: {len(tokens)}')

# b. Generate training data
print("\n[Stage B] Generating training data (Context-Target pairs)...")
X_context = [] # Input: Sum of one-hot vectors of context words
Y_target = [] # Output: One-hot vector of the target word

for i, target_word in enumerate(tokens):
    # Determine context window boundaries
    start_index = max(0, i - CONTEXT_WINDOW_SIZE)
    end_index = min(len(tokens), i + CONTEXT_WINDOW_SIZE + 1)
    context_words = tokens[start_index:i] + tokens[i+1:end_index]

    if not context_words:
        continue

    # Convert context words to indices
    context_indices = [word_to_idx[w] for w in context_words]

    # 1. Input (Context Summation) - We will handle the averaging/summation in the model architecture
    # For now, prepare the input as a list of context word indices
    # (The Embedding layer will handle the lookup)
    X_context.append(context_indices)

    # 2. Output (Target Word)
    target_index = word_to_idx[target_word]
    Y_target.append(target_index)

# Convert lists to NumPy arrays
# X_context must be padded so all input sequences have the same length (4)
MAX_CONTEXT_LEN = 2 * CONTEXT_WINDOW_SIZE
X_context = tf.keras.preprocessing.sequence.pad_sequences(
    X_context, maxlen=MAX_CONTEXT_LEN, padding='post', value=VOCAB_SIZE)
Y_target = np.array(Y_target)

# Convert target labels to one-hot encoding

```

```

Y_target_one_hot = to_categorical(Y_target, num_classes=VOCAB_SIZE)

print(f'Shape of X_context (Input): {X_context.shape}')

print(f'Shape of Y_target (Output): {Y_target_one_hot.shape}')

print(f'Max Context Length (Input Size): {MAX_CONTEXT_LEN}')

# Model Architecture (CBOW is inherently a shallow neural network)

# The CBOW skip-gram structure is often implemented by summing the embeddings of the context words.

print("\n[Stage C - Part 1] Defining Model Architecture...")

# 1. Input layer: Takes padded context indices (e.g., [1, 5, 20, 0])

input_context = Input(shape=(MAX_CONTEXT_LEN,), name='context_input')

# 2. Embedding layer: Converts indices to dense vectors

# mask_zero=True allows the model to ignore the padding value (VOCAB_SIZE used for padding)

embedding_layer = Embedding(

    input_dim=VOCAB_SIZE + 1, # +1 for the padding mask value

    output_dim=EMBEDDING_DIM,

    mask_zero=True,

    name='word_embedding'

)(input_context)

# 3. Summation layer (CBOW magic): Sums the embeddings of the context words

# The Lambda layer applies a custom function (summation over the time/sequence axis)

summed_embeddings = Lambda(lambda x: K.sum(x, axis=1),

output_shape=(EMBEDDING_DIM,))(embedding_layer)

# 4. Hidden Layer (Projection layer - Optional but often used)

projection = Dense(HIDDEN_DIM, activation='relu', name='projection_layer')(summed_embeddings)

# 5. Output layer: Predicts the target word (size = Vocabulary Size)

output_word = Dense(VOCAB_SIZE, activation='softmax', name='output_softmax')(projection)

cbow_model = Model(inputs=input_context, outputs=output_word, name='CBOW_Model')

cbow_model.summary()


# c. Train model

print("\n[Stage C - Part 2] Training Model...")

cbow_model.compile(

    optimizer=Adam(learning_rate=LEARNING_RATE),

```

```

    loss='categorical_crossentropy',
    metrics=['accuracy'])

# Train the model

history = cbow_model.fit(
    X_context,
    Y_target_one_hot,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=0 # Suppress output during actual training)

print(f"Training finished after {EPOCHS} epochs.")

#d.Output

print("\n[Stage D] Outputting Word Embeddings and Sample Prediction...")

# Extract the trained Embedding layer weights

# The weights are the word vectors!

word_vectors = cbow_model.get_layer('word_embedding').get_weights()[0]

print(f"\nExtracted Word Vector Shape: {word_vectors.shape}")

print(f"First 5 dimensions of 'fox': {word_vectors[word_to_idx['fox']][:5]}")

print(f"First 5 dimensions of 'dog': {word_vectors[word_to_idx['dog']][:5]}")


# Example Prediction (Find the most likely word for a context)

sample_context_words = ["the", "quick", "jumps", "over"]

sample_indices = [word_to_idx.get(w, 0) for w in sample_context_words]

# Pad and reshape the input for the model (batch size of 1)

input_data = tf.keras.preprocessing.sequence.pad_sequences(
    [sample_indices], maxlen=MAX_CONTEXT_LEN, padding='post', value=VOCAB_SIZE)

prediction_probabilities = cbow_model.predict(input_data, verbose=0)[0]

predicted_index = np.argmax(prediction_probabilities)

predicted_word = idx_to_word[predicted_index]

print(f"\nContext: {sample_context_words}")

print(f"Predicted Word (Highest Probability): {predicted_word}")

```

Output :

Model: "CBOW_Model"

Layer (type)	Output Shape	Param #	Connected to
context_input (InputLayer)	(None, 4)	0	-
word_embedding (Embedding)	(None, 4, 50)	1,050	context_input[0][0]
not_equal (NotEqual)	(None, 4)	0	context_input[0][0]
lambda (Lambda)	(None, 50)	0	word_embedding[0][0], not_equal[0][0]
projection_layer (Dense)	(None, 128)	6,528	lambda[0][0]
output_softmax (Dense)	(None, 20)	2,580	projection_layer[0][0]

Total params: 10,158 (39.68 KB)

Trainable params: 10,158 (39.68 KB)

Non-trainable params: 0 (0.00 B)

[Stage C - Part 2] Training Model...

Training finished after 100 epochs.

[Stage D] Outputting Word Embeddings and Sample Prediction...

Extracted Word Vector Shape: (21, 50)

First 5 dimensions of 'fox': [-0.07877444 -0.09516871 0.11980705 0.0219985 0.05721781]

First 5 dimensions of 'dog': [-0.1373011 0.0554181 0.02891603 -0.08219764 0.01600051]

Context: ['the', 'quick', 'jumps', 'over']

Predicted Word (Highest Probability): fox

PS D:\Python Programs>

Practical – 6

```
import tensorflow as tf

from tensorflow.keras.applications import VGG16

from tensorflow.keras.models import Model, Sequential

from tensorflow.keras.layers import Dense, Flatten, Dropout, Input

from tensorflow.keras.optimizers import Adam

from tensorflow.keras.utils import to_categorical

from tensorflow.keras.datasets import cifar10

import numpy as np

import matplotlib.pyplot as plt

# --- Configuration ---

IMG_SIZE = 32 # CIFAR-10 images are 32x32

INPUT_SHAPE = (IMG_SIZE, IMG_SIZE, 3)

VGG_INPUT_SIZE = 224 # VGG16 requires 224x224 input

NUM_CLASSES = 10

# Reduced BATCH_SIZE to alleviate memory pressure during fine-tuning (was 32)

BATCH_SIZE = 16

EPOCHS_CLASSIFIER = 2    # <-- EDITED: Reduced from 5 to 2 for speed

EPOCHS_FINETUNE = 2      # <-- EDITED: Reduced from 5 to 2 for speed

print("--- Starting Transfer Learning Implementation (VGG16 on CIFAR-10) ---")

# Data preparation

print("\n[Stage A - Data Preparation] Loading and preprocessing data...")

# Load CIFAR-10 dataset (32x32 color images)

(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Normalize pixel values

x_train = x_train.astype('float32') / 255.0

x_test = x_test.astype('float32') / 255.0

# Convert labels to one-hot encoding

y_train = to_categorical(y_train, NUM_CLASSES)

y_test = to_categorical(y_test, NUM_CLASSES)

# a. Load in a pre-trained CNN model trained on a large dataset

print("\n[Stage A - Base Model] Loading VGG16 (ImageNet weights)...")
```

```

# Instantiate VGG16, excluding the top (classification) layers.
base_model = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(VGG_INPUT_SIZE, VGG_INPUT_SIZE, 3) # Specify VGG's expected input shape here)
# b. Freeze parameters (weights) in the model's lower convolutional layers
print("[Stage B] Freezing VGG16 layers...")
base_model.trainable = False # Freeze all layers in the base model
# c. Add a custom classifier with several layers of trainable parameters
print("[Stage C] Adding custom trainable classifier head...")
# Define the overall model input for our small dataset (32x32x3)
inputs = Input(shape=INPUT_SHAPE)
# 1. Add resizing layer to meet VGG16's expected input size (224x224)
resize_layer = tf.keras.layers.Resizing(VGG_INPUT_SIZE, VGG_INPUT_SIZE, interpolation="bilinear",
name='resizing_layer')
x = resize_layer(inputs)
# 2. Pass resized input through the frozen VGG base model
x = base_model(x, training=False)
# 3. Add new classification layers (head)
x = Flatten()(x)
x = Dense(512, activation='relu')(x)
x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
outputs = Dense(NUM_CLASSES, activation='softmax')(x)
# Create the final model
model = Model(inputs, outputs)
# Compile the model for initial training (only the new layers are trainable)
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy'])
model.summary()

```

```

# d. Train classifier layers on training data available for the task
print(f"\n[Stage D] Training only the custom classifier head (Epochs: {EPOCHS_CLASSIFIER})...")
# Initial training phase (only new layers train)
history_initial = model.fit(
    x_train, y_train,
    epochs=EPOCHS_CLASSIFIER,
    # Use the reduced BATCH_SIZE here
    batch_size=BATCH_SIZE,
    validation_data=(x_test, y_test),
    verbose=1)

# e. Fine-tune hyperparameters and unfreeze more layers as needed
print(f"\n[Stage E] Fine-tuning: Conservatively unfreezing high-level feature blocks (Block 4 & 5)...")
# 1. Unfreeze the base model
base_model.trainable = True
# 2. Select specific layers/blocks to unfreeze (Memory Optimization: Keep more layers frozen)
# Freeze the first three blocks (Block 1, Block 2, Block 3)
for layer in base_model.layers:
    # We only want to train block4 and block5 to save memory
    if layer.name.startswith('block1') or layer.name.startswith('block2') or layer.name.startswith('block3'):
        layer.trainable = False
    else:
        # Unfreeze all layers from block 4 onwards.
        layer.trainable = True

# 3. Recompile the model with a much lower learning rate for fine-tuning
model.compile(
    optimizer=Adam(learning_rate=0.00001), # CRITICAL: Very low LR prevents catastrophic forgetting
    loss='categorical_crossentropy',
    metrics=['accuracy'])
# 4. Continue training (Fine-tuning phase)
print(f"\nFine-tuning entire model (Epochs: {EPOCHS_FINETUNE})...")
history_finetune = model.fit(
    x_train, y_train,

```



```

epochs=EPOCHS_FINETUNE,

# Use the reduced BATCH_SIZE here

batch_size=BATCH_SIZE,

validation_data=(x_test, y_test),

verbose=1)

# Final Evaluation

print("\n--- Final Model Performance Estimation ---")

loss, accuracy = model.evaluate(x_test, y_test, verbose=0)

print(f"Test Loss after Fine-tuning: {loss:.4f}")

print(f"Test Accuracy after Fine-tuning: {accuracy*100:.2f}%")

```

Output :

```

[Stage A - Data Preparation] Loading and preprocessing data...

[Stage A - Base Model] Loading VGG16 (ImageNet weights)...

```

```

[Stage B] Freezing VGG16 layers...
[Stage C] Adding custom trainable classifier head...
Model: "functional"

```

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 32, 32, 3)	0
resizing_layer (Resizing)	(None, 224, 224, 3)	0
vgg16 (Functional)	(None, 7, 7, 512)	14,714,688
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 512)	12,845,568
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 128)	65,664
dense_2 (Dense)	(None, 10)	1,290

```

Total params: 27,627,210 (105.39 MB)
Trainable params: 12,912,522 (49.26 MB)
Non-trainable params: 14,714,688 (56.13 MB)

```

```

[Stage D] Training only the custom classifier head (Epochs: 2)...
Epoch 1/2
3125/3125 ————— 8604s 3s/step - accuracy: 0.3034 - loss: 1.8510 - val_accuracy: 0.5306 - val_loss: 1.5061
Epoch 2/2
3125/3125 ————— 8534s 3s/step - accuracy: 0.3734 - loss: 1.6765 - val_accuracy: 0.5201 - val_loss: 1.4731

[Stage E] Fine-tuning: Conservatively unfreezing high-level feature blocks (Block 4 & 5)...
Epoch 1/2
3125/3125 ————— 14592s 5s/step - accuracy: 0.4727 - loss: 1.3975 - val_accuracy: 0.6303 - val_loss: 1.0742
Epoch 2/2
3125/3125 ————— 14932s 5s/step - accuracy: 0.6163 - loss: 1.0442 - val_accuracy: 0.7745 - val_loss: 0.7315

```

```

--- Final Model Performance Estimation ---
Test Loss after Fine-tuning: 0.7315
Test Accuracy after Fine-tuning: 77.45%

```